Experiment No. : 4

Title: To make basic API requests and handle the responses.

Batch: C3 **Roll No.: 16014225801** **Experiment No.: 5**

Aim: To make basic HTTP API requests using Python's requests library and examine the response received from the server.

Resources Required

- Python 3.x
- Selenium
- VS Code / PyCharm / Jupyter Notebook
- Web browser – Chrome/Edge/Firefox

Target Website: https://api.open-meteo.com

Theory:

An API (Application Programming Interface) allows different software applications to communicate with each other and exchange data. APIs commonly return structured data such as JSON or XML.

In web data acquisition, two common approaches are:

- Web Scraping: extracts information from HTML webpages.
- API: obtains structured data directly from a server.

The lecture notes indicate that APIs are generally faster and more reliable when an appropriate API is available.

Python's Requests library is used to send HTTP requests to web servers. It supports operations such as sending GET and POST requests and receiving server responses.

Basic API Communication

The basic interaction can be represented as:

Python Application → API Endpoint → Server → Response → Python Application

For a GET request:

Client
↓
GET Request
↓
API Server
↓
JSON Response
↓
Python Program

Procedure

1. Import the requests library.
2. Select a publicly accessible API endpoint.
3. Send a GET request using requests.get().
4. Store the server response in a variable.
5. Examine the response status code.
6. Examine the response content.
7. Convert a JSON response into Python data using .json().

8. Extract selected fields from the response.
9. Display the extracted information.

Procedure of Program

1. Import Requests

import requests

The Requests library provides functions for sending HTTP requests.

2. Specify the API endpoint

url = "https://jsonplaceholder.typicode.com/posts/1"

The URL identifies the API resource from which the application requests information.

3. Send a GET request

response = requests.get(url)

requests.get() sends a GET request to the specified endpoint. The returned server response is stored in response. The lecture specifically demonstrates this relationship between requests.get() and the response object.

4. Check the status code

response.status_code

The status_code provides the HTTP status associated with the response. Other response attributes covered in the lecture include text, headers, and url.

5. Read the response

response.text

response.text provides the response content as text.

6. Convert JSON to Python data

data = response.json()

When the API response is JSON, it can be converted into a Python data structure for further processing.

7. Extract required fields

data["userId"]
data["id"]
data["title"]

Individual fields can then be accessed from the returned data.

Expected Output

A typical output will be of the following form:

Status Code: 200

Response:

```
{
  "userId": 1,
  "id": 1,
  "title": "...",
  "body": "..."
}
```

Observation

Parameter	Observation
API request method	GET
Response object	response
HTTP status	200 for a successful request
Response content	JSON/text
JSON conversion	response.json()
Selected fields	userId, id, title

Implementation and Results: (Paste program and screenshots)
CODE:

```
import requests

# 1. Specify the API endpoint
# Location: Kalyan, Maharashtra, India
url = "https://api.open-
meteo.com/v1/forecast?latitude=19.24&longitude=73.13&daily=temperature_2m_max,tempera
ture_2m_min&timezone=auto"

# 2. Send a GET request
response = requests.get(url)

# 3. Check the status code
print("Status Code:", response.status_code)

# 4. Check whether the request was successful
if response.status_code == 200:

    # 5. Read the response
    print("\nResponse:")
    print(response.text)

    # 6. Convert JSON response to Python data
    data = response.json()

    # 7. Extract required information
    dates = data["daily"]["time"]
    max_temperature = data["daily"]["temperature_2m_max"]
    min_temperature = data["daily"]["temperature_2m_min"]

    print("\nClimate Prediction / Heatwave Intelligence")
    print("-----------------------------------------")

    # 8. Display temperature predictions
    for i in range(len(dates)):
```

```
        print("Date:", dates[i])
        print("Maximum Temperature:", max_temperature[i], "°C")
        print("Minimum Temperature:", min_temperature[i], "°C")

        # 9. Basic heatwave detection
        if max_temperature[i] >= 40:
            print("Heatwave Alert: High temperature detected!")
        elif max_temperature[i] >= 37:
            print("Warning: Very High Temperature")
        else:
            print("Status: Normal Temperature")

        print()

else:
    print("Failed to retrieve climate data.")
    print("Error:", response.status_code)
```

Output:

```
Response:{"latitude":19.226713,"longitude":73.08605,"generationtime_ms":0.03182888031
0058594,"utc_offset_seconds":19800,"timezone":"Asia/Kolkata","timezone_abbreviation":
"GMT+5:30","elevation":13.0,"daily_units":{"time":"iso8601","temperature_2m_max":"°C"
,"temperature_2m_min":"°C"},"daily":{"time":["2026-09-21","2026-09-22","2026-09-
23","2026-09-24","2026-09-25","2026-09-26","2026-09-
27"],"temperature_2m_max":[31.8,31.2,31.2,31.8,31.0,30.6,30.4],"temperature_2m_min":[
26.0,25.3,25.2,25.2,25.2,25.8,24.7]}}

Climate Prediction / Heatwave Intelligence----------------------------------------
Date: 2026-09-21
Maximum Temperature: 31.8 °C
Minimum Temperature: 26.0 °C
Status: Normal Temperature
45 |
Date: 2026-09-22
Maximum Temperature: 31.2 °C
Minimum Temperature: 25.3 °C
Status: Normal Temperature
45 |
Date: 2026-09-23
Maximum Temperature: 31.2 °C
Minimum Temperature: 25.2 °C
Status: Normal Temperature
45 |
Date: 2026-09-24
Maximum Temperature: 31.8 °C
Minimum Temperature: 25.2 °C
Status: Normal Temperature
```

```
45 |
Date: 2026-09-25
Maximum Temperature: 31.0 °C
Minimum Temperature: 25.2 °C
Status: Normal Temperature
45 |
Date: 2026-09-26
Maximum Temperature: 30.6 °C
Minimum Temperature: 25.8 °C
Status: Normal Temperature
45 |
Date: 2026-09-27
Maximum Temperature: 30.4 °C
Minimum Temperature: 24.7 °C
Status: Normal Temperature
45 |
Run completed in 5,198.800ms
```

Post-Lab Questions

AI Use Case	How API Requests Fit	What Students Can Demonstrate
Climate Prediction or Heatwave Intelligence	Weather and climate observations can be obtained from APIs and supplied to AI models for prediction or early-warning analysis.	Request weather data → handle JSON response → extract temperature/weather variables.

Conclusion:

The experiment successfully retrieved weather data using a Python API request, processed JSON responses, and analyzed temperatures for heatwave intelligence.